

MongoDB & Mongoose Models - Introduction to MongoDB & NoSQL basics - Mongoose schema & model creation - Connecting Express to MongoDB - Hands-on: Build and test a User model with Mongoose. (Refer to Mongoose docs and MongoDB Atlas setup.)

MongoDB & Mongoose Models

1. Introduction to MongoDB

What is MongoDB?

MongoDB is a **NoSQL database** that stores data in the form of **documents** instead of tables.

SQL databases use:

- Tables
- Rows
- Columns

MongoDB uses:

- Collections
- Documents
- Fields

SQL vs MongoDB

SQL	MongoDB
Table	Collection
Row	Document

SQL**MongoDB**

Column

Field

Example SQL row:

id	name	age
1	Ali	20

MongoDB document:

```
{  
  name: "Ali",
```

```
</> JavaScript
```



Why MongoDB?

- Flexible structure
- Easy to use with JavaScript
- Fast development
- Good for modern web apps
- Stores JSON-like data

2. What is Mongoose?

Mongoose

Mongoose is an ODM (Object Data Modeling) library for Node.js.

It helps us:

- Connect MongoDB with Express/Node
- Create schemas
- Validate data
- Work with database easily

Create project:



5. Create Express Server

server.js

```
const express = require("express");

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.send("Server Running");
});

app.listen(3000, () => {
```

⌄ JavaScript



Run server:

⌄ Bash



OR

⌄ Bash



6. MongoDB Atlas Setup

MongoDB Atlas

MongoDB Atlas is MongoDB's cloud database service.

Steps:

1. Create account on [MongoDB Atlas ↗](#)
2. Create cluster

3. Create database user

4. Whitelist IP

Use:

0.0.0.0/0



5. Click "Connect"

6. Copy connection string

Example:

mongodb+srv://username:password@cluster0.mongodb.net/myDB



7. Connect Express to MongoDB

Update server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();

app.use(express.json());

mongoose.connect(
  "YOUR_MONGODB_CONNECTION_STRING"
)
.then(() => {
  console.log("MongoDB Connected");
})
.catch((err) => {
  console.log(err);
});

app.get("/", (req, res) => {
  res.send("Server Running");
});

app.listen(3000, () => {
```

</> JavaScript



```
console.log("Server started on port 3000");  
});
```

8. Mongoose Schema

What is Schema?

Schema defines:

- Structure of data
- Data types
- Validation rules

Example:

```
name: String
```

</> JavaScript



9. Create User Model

models/User.js

```
const mongoose = require("mongoose");  
  
const userSchema = new mongoose.Schema({  
  name: {  
    type: String,  
    required: true,  
  },  
  
  email: {  
    type: String,  
    required: true,  
    unique: true,  
  },  
  
  age: {
```

```
    type: Number,  
  },  
});  
  
const User = mongoose.model("User", userSchema);  
  
</> JavaScript
```



10. Understanding Model

```
</> JavaScript
```



This creates a model named:

```
User
```



MongoDB automatically creates collection:

```
users
```



11. Create User Data

Update server.js

```
const express = require("express");  
const mongoose = require("mongoose");  
const User = require("../models/User");  
  
const app = express();  
  
app.use(express.json());  
  
mongoose.connect(  
  "YOUR_MONGODB_CONNECTION_STRING"  
)  
  .then(() => {
```

```
    console.log("MongoDB Connected");
  })
  .catch((err) => {
    console.log(err);
  });

app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);

    await user.save();

    res.status(201).json(user);
  } catch (error) {
    res.status(500).json({
      message: error.message,
    });
  }
});

app.get("/", (req, res) => {
  res.send("Server Running");
});

app.listen(3000, () => {
  //
});
```

JavaScript



12. Test API

Use:

- Postman
- Thunder Client
- Insomnia

POST Request

URL

http://localhost:3000/users



Body

```
{  
  "name": "Ali",  
  "email": "ali@gmail.com",
```

</> JSON



13. Output

Response:

```
{  
  "_id": "664...",  
  "name": "Ali",  
  "email": "ali@gmail.com",  
  "age": 20,
```

</> JSON



14. How Flow Works

Step-by-step Flow

Step 1

User sends request:

POST /users



Step 2

Express receives request:

```
</> JavaScript
```



Step 3

Data comes in:

```
</> JavaScript
```



Step 4

New user object created:

```
</> JavaScript
```



Step 5

Save data in MongoDB:

```
</> JavaScript
```



Step 6

MongoDB stores document in collection:

```
users
```



Step 7

Response sent back:

 JavaScript



15. Simple One-File Version

If you want everything in one file:

server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();

app.use(express.json());

mongoose.connect("YOUR_MONGODB_CONNECTION_STRING")
  .then(() => {
    console.log("MongoDB Connected");
  });

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number,
});

const User = mongoose.model("User", userSchema);

app.post("/users", async (req, res) => {
  const user = new User(req.body);

  await user.save();

  res.json(user);
});

app.listen(3000, () => {
```

`</> JavaScript`

16. Important Mongoose Methods

Save Data

`</> JavaScript`

Find All Users

`</> JavaScript`

Find One User

`</> JavaScript`

Update User

`</> JavaScript`

Delete User

</> JavaScript



17. Common Interview Questions

Difference between MongoDB and Mongoose?

MongoDB

Database

Mongoose

Library to interact with MongoDB

What is a Schema?

Blueprint/structure of data

What is a Model?

Tool used to interact with collection

What is Collection?

Group of documents

What is Document?

Single data object inside collection

       Sources

To-Do CRUD API - Create, Read, Update, Delete operations with Express + Mongoose - Error handling & validation - Hands-on: Build full CRUD for a to-do item with validation middleware. (Use Postman to test routes.(its too much long and difficult „plz simple and easy dw „,also explain code)

To-Do CRUD API (Simple & Easy)

What is CRUD?

CRUD means:

Letter	Meaning
C	Create
R	Read
U	Update
D	Delete

Example in To-Do App:

- Create → Add task
- Read → Show tasks
- Update → Edit task
- Delete → Remove task

1. Install Packages

</> Bash



</> Bash



2. Create server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();

app.use(express.json());

// MongoDB Connection
mongoose.connect("YOUR_MONGODB_URL")
  .then(() => {
    console.log("MongoDB Connected");
  });

// Schema
const todoSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
  },

  completed: {
    type: Boolean,
    default: false,
  },
});

// Model
const Todo = mongoose.model("Todo", todoSchema);

// CREATE
app.post("/todos", async (req, res) => {
  try {
    const todo = new Todo(req.body);

    await todo.save();
```

```
    res.json(todo);

  } catch (error) {
    res.status(500).json({
      message: error.message,
    });
  }
});

// READ
app.get("/todos", async (req, res) => {
  try {
    const todos = await Todo.find();

    res.json(todos);

  } catch (error) {
    res.status(500).json({
      message: error.message,
    });
  }
});

// UPDATE
app.put("/todos/:id", async (req, res) => {
  try {
    const updatedTodo = await Todo.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );

    res.json(updatedTodo);

  } catch (error) {
    res.status(500).json({
      message: error.message,
    });
  }
});

// DELETE
app.delete("/todos/:id", async (req, res) => {
  try {
    await Todo.findByIdAndDelete(req.params.id);
```



```
res.json({
  message: "Todo Deleted",
});

} catch (error) {
  res.status(500).json({
    message: error.message,
  });
}
});
```

```
app.listen(3000, () => {
```

</> JavaScript



3. Run Server

</> Bash



4. Test in Postman

CREATE Todo

Method

POST



URL

http://localhost:3000/todos



Body

{

</> JSON



READ Todos

Method

GET



URL

`http://localhost:3000/todos`

UPDATE Todo

Method

PUT



URL

`http://localhost:3000/todos/PASTE_ID_HERE`

Body

{

</> JSON



DELETE Todo

Method

DELETE



URL

http://localhost:3000/todos/PASTE_ID_HERE



Simple Code Explanation

1. Express Setup

</> JavaScript



Express import kiya.

</> JavaScript



App create ki.

</> JavaScript



JSON data read karne ke liye.

2. MongoDB Connection

</> JavaScript



MongoDB connect karta hai.

3. Schema

</> JavaScript



Data structure banaya.

```
title: {  
  type: String,
```

</> JavaScript



Task title zaruri hai.

```
completed: {  
  type: Boolean,
```

</> JavaScript



Default value false hogi.

4. Model

```
</> JavaScript
```



Model database ke sath work karta hai.

5. CREATE

```
</> JavaScript
```



New todo create karta hai.

```
</> JavaScript
```



New object bana.

```
</> JavaScript
```



Database me save hua.

6. READ

```
</> JavaScript
```



Saare todos laata hai.

7. UPDATE

</> JavaScript



ID se todo update karta hai.

8. DELETE

</> JavaScript



Todo delete karta hai.

Flow of CRUD

Create

Client → Express → MongoDB → Response

Read

Client → Express → MongoDB se data → Response

Update

Client → ID + New Data → MongoDB Update → Response

Delete

Client → ID → MongoDB Delete → Response



give one more code statementr of CRUD

Another Simple CRUD Example (Notes API)

server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();

app.use(express.json());

// MongoDB Connection
mongoose.connect("YOUR_MONGODB_URL")
  .then(() => {
    console.log("MongoDB Connected");
  });

// Schema
const noteSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
  },

  content: {
    type: String,
    required: true,
  },
});

// Model
const Note = mongoose.model("Note", noteSchema);

// CREATE
app.post("/notes", async (req, res) => {
  const note = await Note.create(req.body);

  res.json(note);
});
```

```
});

// READ
app.get("/notes", async (req, res) => {
  const notes = await Note.find();

  res.json(notes);
});

// UPDATE
app.put("/notes/:id", async (req, res) => {
  const updatedNote = await Note.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );

  res.json(updatedNote);
});

// DELETE
app.delete("/notes/:id", async (req, res) => {
  await Note.findByIdAndDelete(req.params.id);

  res.json({
    message: "Note Deleted",
  });
});

app.listen(3000, () => {
```

JavaScript



Postman Testing

CREATE

POST

http://localhost:3000/notes



Body:

```
{  
  "title": "MongoDB",
```

</> JSON



READ

GET

http://localhost:3000/notes



UPDATE

PUT

http://localhost:3000/notes/ID_HERE



Body:

```
{
```

</> JSON



DELETE

DELETE



ChatGPT ▾

 Free offer



Easy Explanation

CREATE

</> JavaScript



New note database me add karta hai.

READ

</> JavaScript



Saari notes show karta hai.

UPDATE

</> JavaScript



```
Note.findByIdAndUpdate()
```

ID se note update karta hai.

DELETE

</> JavaScript



```
Note.findByIdAndDelete()
```

ID se note delete karta hai.



